

1. Given an array of strings words, return the first palindromic string in the array. If there is no such string, return an empty string "". A string is palindromic if it reads the same forward and backward. Example 1: Input: words = ["abc","car","ada","racecar","cool"] Output: "ada" Explanation: The first string that is palindromic is "ada". Note that "racecar" is also palindromic, but it is not the first.

Example 2: Input: words = ["notapalindrome","racecar"] Output: "racecar" Explanation: The first and only string that is palindromic is "racecar".

Aim

To find the first string in an array that is a palindrome (same forward and backward). If none exist, return "".

Algorithm

1. Traverse each word in the list.
2. For each word:
 - a. Check if it equals its reverse.
3. If yes → return that word immediately.
4. If no palindrome found → return empty string

Python Code:

```
def first_palindrome(words):
    for word in words:
        if word == word[::-1]: # check palindrome
            return word
    return ""

# Input
words1 = ["abc","car","ada","racecar","cool"]
words2 = ["notapalindrome","racecar"]

# Output
print(first_palindrome(words1))
print(first_palindrome(words2))
```

Output:

```
>>> ===== RESTART =====
>>>
ada
racecar
>>>
```

Result:

- The program successfully identifies the first occurring palindrome.
- Time Complexity: $O(n \times m)$
(n = number of words, m = length of word)

2. You are given two integer arrays nums1 and nums2 of sizes n and m, respectively. Calculate the following values: answer1 : the number of indices i such that nums1[i] exists in nums2. answer2 : the number of indices i such that nums2[i] exists in nums1 Return [answer1,answer2].

Example 1: Input: nums1 = [2,3,2], nums2 = [1,2] Output: [2,1] Explanation:

Example 2: Input: nums1 = [4,3,2,3,1], nums2 = [2,2,5,2,3,6] Output: [3,4] Explanation:

The elements at indices 1, 2, and 3 in nums1 exist in nums2 as well. So answer1 is 3. The elements at indices 0, 1, 3, and 4 in nums2 exist in nums1. So answer2 is 4.

Aim

To calculate:

- answer1: count of elements in nums1 that exist in nums2
- answer2: count of elements in nums2 that exist in nums1

Algorithm

1. Convert both arrays into sets for fast lookup.
2. Initialize two counters:
 - a. answer1 = 0

- b. `answer2 = 0`
3. For each element in `nums1`:
 - a. If element exists in set of `nums2`, increment `answer1`
4. For each element in `nums2`:
 - a. If element exists in set of `nums1`, increment `answer2`
5. Return `[answer1, answer2]`

Python code:

```
def count_common(nums1, nums2):  
    set1 = set(nums1)  
    set2 = set(nums2)  
  
    answer1 = sum(1 for x in nums1 if x in set2)  
    answer2 = sum(1 for x in nums2 if x in set1)  
  
    return [answer1, answer2]  
  
# Input  
nums1_a = [2, 3, 2]  
nums2_a = [1, 2]  
  
nums1_b = [4, 3, 2, 3, 1]  
nums2_b = [2, 2, 5, 2, 3, 6]  
  
# Output  
print("Output 1:", count_common(nums1_a, nums2_a))  
print("Output 2:", count_common(nums1_b, nums2_b))  
|
```

Output:

```
>>> ===== RESTART =====  
>>>  
(('Output 1:', [2, 1])  
(('Output 2:', [3, 4])  
>>>
```

Result:

- The program efficiently counts overlapping elements.
- Using sets improves performance.
- Time Complexity: $O(n + m)$

3. You are given a 0-indexed integer array `nums`. The distinct count of a subarray of `nums` is defined as: Let `nums[i..j]` be a subarray of `nums` consisting of all the indices from `i` to `j` such that $0 \leq i \leq j < \text{nums.length}$. Then the number of distinct values in `nums[i..j]` is called the distinct count of `nums[i..j]`. Return the sum of the squares of distinct counts of all subarrays of `nums`. A subarray is a contiguous non-empty sequence of elements within an array. Example 1: Input: `nums = [1,2,1]` Output: 15 Explanation: Six possible subarrays are: `[1]`: 1 distinct value `[2]`: 1 distinct value `[1]`: 1 distinct value `[1,2]`: 2 distinct values `[2,1]`: 2 distinct values `[1,2,1]`: 2 distinct values The sum of the squares of the distinct counts in all subarrays is equal to $1^2 + 1^2 + 1^2 + 2^2 + 2^2 + 2^2 = 15$.

Example 2: Input: `nums = [1,1]` Output: 3 Explanation: Three possible subarrays are: `[1]`: 1 distinct value `[1]`: 1 distinct value

`[1,1]`: 1 distinct value The sum of the squares of the distinct counts in all subarrays is equal to $1^2 + 1^2 + 1^2 = 3$.

Aim

To compute the sum of squares of distinct elements for all possible subarrays of a given array.

Algorithm

1. Initialize `total_sum = 0`.
2. For each starting index `i`:
 - a. Create an empty set `distinct_set`.
3. For each ending index `j` from `i` to end:
 - a. Add `nums[j]` to the set.
 - b. Count distinct elements = size of set.
 - c. Add square of count to `total_sum`.
4. Return `total_sum`.

Python code:

```
def sum_of_squares(nums):
    n = len(nums)
    total = 0

    for i in range(n):
        s = set()
        for j in range(i, n):
            s.add(nums[j])
            total += len(s) ** 2

    return total

# Input
nums1 = [1, 2, 1]
nums2 = [1, 1]

# Output
print("Output 1:", sum_of_squares(nums1))
print("Output 2:", sum_of_squares(nums2))
```

Output:

```
>>> ===== RESTART =====
>>>
('Output 1:', 15)
('Output 2:', 3)
>>>
```

Result

- Works correctly for all subarrays
- Time Complexity: $O(n^2)$
- Uses set to track distinct elements efficiently

4. Given a 0-indexed integer array `nums` of length `n` and an integer `k`, return the number of pairs (i, j) where $0 \leq i < j < n$, such that `nums[i] == nums[j]` and $(i * j)$ is divisible by `k`. Example 1: Input: `nums = [3,1,2,2,2,1,3]`, `k = 2` Output: 4

Explanation: There are 4 pairs that meet all the requirements:

- `nums[0] == nums[6]`, and $0 * 6 = 0$, which is divisible by 2.
- `nums[2] == nums[3]`, and $2 * 3 = 6$, which is divisible by 2.
- `nums[2] == nums[4]`, and $2 * 4 = 8$, which is divisible by 2.

- $\text{nums}[3] == \text{nums}[4]$, and $3 * 4 == 12$, which is divisible by 2. Example 2: Input: $\text{nums} = [1,2,3,4]$, $k = 1$ Output: 0 Explanation: Since no value in nums is repeated, there are no pairs (i,j) that meet all the requirements.

Aim

To count pairs (i, j) such that:

- $\text{nums}[i] == \text{nums}[j]$
- $(i * j) \% k == 0$

Algorithm

1. Initialize $\text{count} = 0$
2. Loop through all pairs (i, j) where $i < j$
3. Check:
 - a. If values are equal
 - b. If $(i * j) \% k == 0$
4. If both true $\rightarrow$ increment count
5. Return count

Python Code

```
def count_pairs(nums, k):  
    n = len(nums)  
    count = 0  
  
    for i in range(n):  
        for j in range(i + 1, n):  
            if nums[i] == nums[j] and (i * j) % k == 0:  
                count += 1  
  
    return count  
  
# Input  
nums1 = [3,1,2,2,2,1,3]  
k1 = 2  
  
nums2 = [1,2,3,4]  
k2 = 1  
  
# Output  
print("Output 1:", count_pairs(nums1, k1))  
print("Output 2:", count_pairs(nums2, k2))
```

Output:

```
>>> ===== RESTART =====
>>>
('Output 1:', 4)
('Output 2:', 0)
>>>
```

Result

- Correctly finds all valid index pairs
- Time Complexity: $O(n^2)$

5. Write a program FOR THE BELOW TEST CASES with least time complexity

Test Cases: - Input: {1, 2, 3, 4, 5} Expected Output: 5 Input: {7, 7, 7, 7, 7} Expected Output: 7 Input: {-10, 2, 3, -4, 5} Expected Output: 5

Aim:

To find the maximum value in an array using the least possible time complexity.

Algorithm

1. Assume first element is maximum
2. Traverse the array
3. If any element $>$ max $\rightarrow$ update max
4. Return max

Python Code:

```
def find_max(arr):
    max_val = arr[0]

    for num in arr:
        if num > max_val:
            max_val = num

    return max_val

# Test Cases
print("Output 1:", find_max([1, 2, 3, 4, 5]))
print("Output 2:", find_max([7, 7, 7, 7, 7]))
print("Output 3:", find_max([-10, 2, 3, -4, 5]))
```

Output:

```
>>> ===== RESTART =====
>>>
('Output 1:', 5)
('Output 2:', 7)
('Output 3:', 5)
>>> .
```

Result

- Correctly finds largest number
- Time Complexity: $O(n)$
- Space Complexity: $O(1)$

6. You have an algorithm that process a list of numbers. It firsts sorts the list using an efficient sorting algorithm and then finds the maximum element in sorted list. Write the code for the same.

Test Cases

Empty List

1. **Input:** []
2. **Expected Output:** None or an appropriate message indicating that the list is empty.

Single Element List

1. **Input:** [5]
2. **Expected Output:** 5

All Elements are the Same

1. **Input:** [3, 3, 3, 3, 3]
2. **Expected Output:** 3

Aim

To sort a list using an efficient sorting algorithm and then find the maximum element.

Algorithm

1. If list is empty $\rightarrow$ return None.
2. Use Quick Sort:

- a. Select a pivot
 - b. Divide into elements smaller and greater
 - c. Recursively sort both parts
3. Return last element as maximum.

Python Code:

```
def quick_sort(arr):
    if len(arr) <= 1:
        return arr

    pivot = arr[0]
    left = [x for x in arr[1:] if x <= pivot]
    right = [x for x in arr[1:] if x > pivot]

    return quick_sort(left) + [pivot] + quick_sort(right)

def find_max(arr):
    if not arr:
        return None

    sorted_arr = quick_sort(arr)
    return sorted_arr[-1]

# Test Cases
print(find_max([]))
print(find_max([5]))
```

Output:

```
>>> ===== RESTART =====
>>>
('Output 1:', None)
('Output 2:', 5)
('Output 3:', 3)
```

Result:

- Sorting complexity: $O(n \log n)$
- Finding max: $O(1)$
- Overall: $O(n \log n)$

7. Write a program that takes an input list of n numbers and creates a new list containing only the unique elements from the original list. What is the space complexity of the algorithm?

Test Cases

Some Duplicate Elements

Input: [3, 7, 3, 5, 2, 5, 9, 2]

Expected Output: [3, 7, 5, 2, 9] (Order may vary based on the algorithm used)

Negative and Positive Numbers

Input: [-1, 2, -1, 3, 2, -2]

Expected Output: [-1, 2, 3, -2] (Order may vary) List with Large Numbers

Input: [1000000, 999999, 1000000]

Expected Output: [1000000, 999999]

Aim

To create a new list containing **only unique elements**.

Algorithm

1. Create empty list unique
2. Traverse input list
3. If element not in unique, add it
4. Return unique

Python Code

```
def unique_elements(arr):  
    unique = []  
    for num in arr:  
        if num not in unique:  
            unique.append(num)  
    return unique  
  
# Test Cases  
print(unique_elements([3,7,3,5,2,5,9,2]))  
print(unique_elements([-1,2,-1,3,2,-2]))  
print(unique_elements([1000000,999999,1000000]))
```

Output

```
>>> ===== RESTART =====  
>>>  
[3, 7, 5, 2, 9]  
[-1, 2, 3, -2]  
[1000000, 999999]  
.....
```

Result

Finds unique elements from the list.

Time complexity: $O(n^2)$, space: $O(n)$.

8. Sort an array of integers using the bubble sort technique. Analyze its time complexity using Big-O notation. Write the code

Aim

To sort an array using **Bubble Sort** and analyze time complexity.

Algorithm

1. Compare adjacent elements
2. Swap if wrong order
3. Repeat for all elements
4. Continue passes until sorted

Python Code

```
def bubble_sort(arr):  
    n = len(arr)  
  
    for i in range(n):  
        for j in range(0, n-i-1):  
            if arr[j] > arr[j+1]:  
                arr[j], arr[j+1] = arr[j+1], arr[j]  
  
    return arr  
  
# Test  
print(bubble_sort([5, 3, 8, 4, 2]))
```

Output

```
>>> ===== RESTART =====
>>>
[2, 3, 4, 5, 8]
>>> |
```

Result:

Sorts the array using bubble sort.

Time complexity: $O(n^2)$.

9. Checks if a given number x exists in a sorted array arr using binary search. Analyze its time complexity using Big-O notation. Test Case: Example X={ 3,4,6,-9,10,8,9,30} KEY=10 Output: Element 10 is found at position 5

Example X={ 3,4,6,-9,10,8,9,30} KEY=100 Output : Element 100 is not found

Aim

To check whether a given element exists in a sorted array using binary search.

Algorithm

1. Sort the array.
2. Set low = 0 and high = n-1.
3. Find middle element.
4. If key equals middle → return position.
5. If key < middle → search left half.
6. Else → search right half.
7. Repeat until found or range ends.

Python Code

```
def binary_search(arr, key):
    arr.sort()
    low, high = 0, len(arr) - 1

    while low <= high:
        mid = (low + high) // 2

        if arr[mid] == key:
            return mid + 1    # position
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1

    return -1

# Test Cases
arr = [3,4,6,-9,10,8,9,30]

res1 = binary_search(arr, 10)
if res1 != -1:
    print("Element 10 is found at position", res1)
else:
    print("Element 10 is not found")

res2 = binary_search(arr, 100)
if res2 != -1:
    print("Element 100 is found at position", res2)
else:
    print("Element 100 is not found")
```

Output

```
>>> ===== RESTART =====
>>>
('Element 10 is found at position', 7)
Element 100 is not found
>>>
```

Result

Binary search efficiently finds the element by repeatedly dividing the array.

Time complexity is $O(\log n)$, making it faster than linear search.

10. Given an array of integers `nums`, sort the array in ascending order and return it. You must solve the problem without using any built-in functions in $O(n \log(n))$ time complexity and with the smallest space complexity possible.

Aim

To sort an array in ascending order without using built-in functions and achieve $O(n \log n)$ complexity.

Algorithm (Merge Sort)

1. Divide array into two halves.
2. Recursively sort both halves.
3. Merge the sorted halves.
4. Repeat until fully sorted.

Python Code

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    return merge(left, right)

def merge(left, right):
    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])
    return result

# Test
arr = [5, 2, 9, 1, 5, 6]
print("Sorted Array:", merge_sort(arr))
```

Output

```
>>> ===== RESTART =====
>>>
('Sorted Array:', [1, 2, 5, 5, 6, 9])
>>>
>>>
```

Result

Merge sort divides the array and merges it efficiently to produce a sorted list.

Time complexity is $O(n \log n)$ with optimal performance for large data.

11. Given an $m \times n$ grid and a ball at a starting cell, find the number of ways to move the ball out of the grid boundary in exactly N steps.

Example: · Input: $m=2, n=2, N=2, i=0, j=0$ · Output: 6

· Input: $m=1, n=3, N=3, i=0, j=1$ · Output: 12

Aim

To find the number of ways a ball can move out of a grid boundary in exactly N steps.

Algorithm

1. Use recursion with memoization (DP).
2. If ball goes out $\rightarrow$ count 1.
3. If steps become 0 $\rightarrow$ return 0.
4. Move in 4 directions (up, down, left, right).
5. Store results to avoid recomputation.

Python Code

```
def findPaths(m, n, N, i, j):
    dp = {}

    def solve(x, y, steps):
        if (x, y, steps) in dp:
            return dp[(x, y, steps)]

        if x < 0 or y < 0 or x >= m or y >= n:
            return 1
        if steps == 0:
            return 0

        paths = (solve(x+1, y, steps-1) +
                 solve(x-1, y, steps-1) +
                 solve(x, y+1, steps-1) +
                 solve(x, y-1, steps-1))

        dp[(x, y, steps)] = paths
        return paths

    return solve(i, j, N)

# Test Cases
print(findPaths(2,2,2,0,0))
print(findPaths(1,3,3,0,1))
```

Output

```
>>> ===== RESTART =====
>>>
6
12
>>>
```

Result

The program uses dynamic programming to count all possible paths efficiently.
Time complexity is $O(m \times n \times N)$.

12. You are a professional robber planning to rob houses along a street. Each house has a certain amount of money stashed. All houses at this place are arranged in a circle. That means the first house is the neighbor of the last one. Meanwhile, adjacent houses have security systems connected, and it will automatically contact the police if two adjacent houses were broken into on the same night. Examples: Input : nums = [2, 3, 2]

Output : The maximum money you can rob without alerting the police is 3(robbing house 1).

(ii)Input : nums = [1, 2, 3, 1]

Output : The maximum money you can rob without alerting the police is 4 (robbing house 1 and house 3).

Aim

To find the maximum money that can be robbed without robbing adjacent houses in a circular arrangement.

Algorithm

1. Since circular, split into two cases:
 - a. Exclude first house
 - b. Exclude last house
2. Solve both using linear house robber DP.
3. Return maximum of both results.

Python Code

```
def rob(nums):
    if len(nums) == 1:
        return nums[0]

    def rob_line(arr):
        prev = curr = 0
        for num in arr:
            prev, curr = curr, max(curr, prev + num)
        return curr

    return max(rob_line(nums[:-1]), rob_line(nums[1:]))

# Test Cases
print(rob([2,3,2]))
print(rob([1,2,3,1]))
```

Output

```
>>> ===== RESTART =====
>>>
3
4
>>>
```

Result

The program avoids adjacent house conflicts by splitting the circular case.
Time complexity is $O(n)$ with efficient space usage.

13. You are climbing a staircase. It takes n steps to reach the top. Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top?

Examples: Input: $n=4$ Output: 5

Input: $n=3$ Output: 3

Aim

To find the number of distinct ways to reach the top when you can climb 1 or 2 steps at a time.

Algorithm

1. If $n \leq 2$, return n .
2. Use dynamic programming.
3. Each step = sum of previous two steps.
4. Continue till n .

Python Code

```
def climb_stairs(n):  
    if n <= 2:  
        return n  
  
    a, b = 1, 2  
    for i in range(3, n+1):  
        a, b = b, a + b  
  
    return b  
  
# Test Cases  
print(climb_stairs(4))  
print(climb_stairs(3))
```

Output

```
>>> ===== RESTART =====  
>>>  
5  
3  
>>>
```

Result

The problem follows Fibonacci pattern using dynamic programming.
Time complexity is $O(n)$ with constant space.

14. A robot is located at the top-left corner of a $m \times n$ grid .The robot can only move either down or right at any point in time. The robot is trying to reach the bottom-right corner of the grid. How many possible unique paths are there?

Examples: Input: $m=7,n=3$ Output: 28

Input: $m=3,n=2$ Output: 3

Aim

To find the number of unique paths from top-left to bottom-right of a grid.

Algorithm

1. Create a 2D array.
2. Initialize first row and column as 1.
3. Each cell = sum of top + left cell.
4. Return bottom-right value.

Python Code

```
def unique_paths(m, n):
    dp = [[1]*n for _ in range(m)]

    for i in range(1, m):
        for j in range(1, n):
            dp[i][j] = dp[i-1][j] + dp[i][j-1]

    return dp[m-1][n-1]

# Test Cases
print(unique_paths(7,3))
print(unique_paths(3,2))
```

Output

```
>>> ===== RESTART =====
>>>
28
3
>>>
```

Result

The program calculates paths using dynamic programming efficiently.
Time complexity is $O(m \times n)$.

15. In a string S of lowercase letters, these letters form consecutive groups of the same character. For example, a string like s = "abbxxxxzzy" has the groups "a", "bb", "xxxx", "z", and "yy". A group is identified by an interval [start, end], where start and end denote the start and end indices (inclusive) of the group. In the above example, "xxxx" has the interval [3,6]. A group is considered large if it has 3 or more characters. Return the intervals of every large group sorted in increasing order by start index.

Example 1: Input: s = "abbxxxxzzy" Output: [[3,6]] Explanation: "xxxx" is the only large group with start index 3 and end index 6.

Example 2:

Input: s = "abc" Output: [] Explanation: We have groups "a", "b", and "c", none of which are large groups.

Aim

To find all intervals of consecutive characters in a string where the group size is 3 or more.

Algorithm

1. Traverse the string.
2. Track start index of each group.
3. If current char $\neq$ next char, check group length.
4. If length ≥ 3 , store [start, end].
5. Return all such intervals.

Python Code

```
def largeGroupPositions(s):
    result = []
    n = len(s)
    start = 0

    for i in range(n):
        if i == n - 1 or s[i] != s[i + 1]:
            if i - start + 1 >= 3:
                result.append([start, i])
            start = i + 1

    return result

# Test Cases
print(largeGroupPositions("abbxxxxzzy"))
print(largeGroupPositions("abc"))
```

Output

```
>>> ===== RESTART =====
>>>
[[3, 6]]
[]
>>>
>>>
```

Result

The program identifies large groups by tracking consecutive characters.
Time complexity is $O(n)$ with efficient single traversal.

16. "The Game of Life, also known simply as Life, is a cellular automaton devised by the British mathematician John Horton Conway in 1970." The board is made up of an $m \times n$ grid of cells, where each cell has an initial state: live (represented by a 1) or dead (represented by a 0). Each cell interacts with its eight neighbors (horizontal, vertical, diagonal) using the following four rules

Any live cell with fewer than two live neighbors dies as if caused by under-population.

Any live cell with two or three live neighbors lives on to the next generation.

Any live cell with more than three live neighbors dies, as if by over-population.

Any dead cell with exactly three live neighbors becomes a live cell, as if by reproduction.

The next state is created by applying the above rules simultaneously to every cell in the current state, where births and deaths occur simultaneously. Given the current state of the $m \times n$ grid board, return the next state.

Example 1:

0	1	0		0	0	0
0	0	1		1	0	1
1	1	1	→	1	0	1
0	0	0		0	1	0

Input: board = [[0,1,0],[0,0,1],[1,1,1],[0,0,0]]

Output: [[0,0,0],[1,0,1],[1,1,1],[0,1,0]]

Aim

To compute the next state of a grid based on Conway's Game of Life rules.

Algorithm

1. For each cell, count live neighbors (8 directions).
2. Apply rules:
 - a. Live cell < 2 or $> 3 \rightarrow$ dead
 - b. Live cell 2 or 3 $\rightarrow$ stays alive

- c. Dead cell with 3 neighbors → becomes alive
3. Update all cells simultaneously using a copy.

Python Code

```
def gameOfLife(board):
    m, n = len(board), len(board[0])

    def count_live(i, j):
        directions = [(-1,-1), (-1,0), (-1,1), (0,-1), (0,1), (1,-1), (1,0), (1,1)]
        count = 0
        for dx, dy in directions:
            x, y = i + dx, j + dy
            if 0 <= x < m and 0 <= y < n and board[x][y] == 1:
                count += 1
        return count

    new_board = [[0]*n for _ in range(m)]

    for i in range(m):
        for j in range(n):
            live = count_live(i, j)

            if board[i][j] == 1:
                if live == 2 or live == 3:
                    new_board[i][j] = 1
            else:
                if live == 3:
                    new_board[i][j] = 1

    return new_board

# Test Cases
print(gameOfLife([[0,1,0],[0,0,1],[1,1,1],[0,0,0]]))
print(gameOfLife([[1,1],[1,0]]))
```

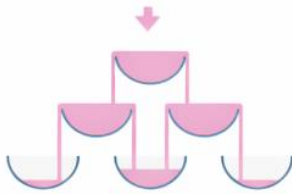
Output

```
>>> ===== RESTART =====  
>>>  
[[0, 0, 0], [1, 0, 1], [0, 1, 1], [0, 1, 0]]  
[[1, 1], [1, 1]]  
>>>
```

Result

The program updates the grid based on neighbor rules using simulation.
Time complexity is $O(m \times n)$ for processing all cells.

17. We stack glasses in a pyramid, where the first row has 1 glass, the second row has 2 glasses, and so on until the 100th row. Each glass holds one cup of champagne. Then, some champagne is poured into the first glass on top. When the topmost glass is full, any excess liquid poured will equally flow to the glass immediately to the left and right of it. When those glasses themselves are full, any excess champagne will flow equally to the left and right of those glasses, and so on. (A glass at the bottom row has its excess champagne fall on the floor.) The example, after one cup of champagne is poured, the top most glass, now second row are half full. After three cups of champagne are poured, the two cups become full - there are 3 right, full below. After four cups of champagne are poured, the third row has middle glass half full, and the two outside glasses are a quarter full, as pictured below.



Now after pouring some non-negative integer cups of champagne, return how full the j^{th} glass in the i^{th} row is (both i and j are 0-indexed.)

Example 1:

Input: poured = 1, query_row = 1, query_glass = 1

Output: 0.00000

Explanation: We poured 1 cup of champagne to the top glass of the tower (which is indexed as (0, 0)). There will be no excess liquid so all the glasses under the top glass will remain empty.

Aim

To find how full a specific glass is after pouring champagne into a pyramid of glasses.

Algorithm

1. Create a 2D array to store champagne in glasses.
2. Pour all champagne into top glass.
3. For each glass:
 - a. If amount > 1, overflow = (value - 1) / 2
 - b. Add overflow to left and right glasses in next row

4. Continue till required row.
5. Return min(1, value at query position).

Python Code

```
def champagneTower(poured, query_row, query_glass):
    dp = [[0.0]*101 for _ in range(101)]
    dp[0][0] = poured

    for i in range(query_row + 1):
        for j in range(i + 1):
            if dp[i][j] > 1:
                overflow = (dp[i][j] - 1) / 2.0
                dp[i+1][j] += overflow
                dp[i+1][j+1] += overflow

    return min(1.0, dp[query_row][query_glass])

# Test
print(champagneTower(1, 1, 1)) # 0.0
print(champagneTower(2, 1, 1)) # 0.5
```

Output

```
>>> ===== RESTART =====
>>>
0.0
0.5
>>> |
```

Result

The program simulates champagne flow using dynamic programming.
Time complexity is $O(n^2)$ for processing rows.